Guia Prático & Atividade de Laboratório — Aula 04

Integração Externa com TMDB: RestClient, DTOs Externos, Anti-Corruption Layer & Segredos

Módulo: Java Backend Developer	**Tecnologias:** Java 21, Spring Boot 4.1.x, RestClient, Jackson
Duração Prevista: 45 minutos	**Checkpoint Git:** checkpoint/aula-04-tmdb

1. Resumo Teórico & Dicas de Arquitetura (Guia de Estudo)

Na Aula 03, estabelecemos a persistência local da nossa aplicação com Spring Data JPA, Hibernate e Banco H2. No entanto, um catálogo de filmes necessita de dados ricos e atualizados (títulos reais, anos de lançamento, avaliações e imagens de poster). Na Aula 04, integramos a nossa API MyMovies a um fornecedor externo de dados: a API REST pública do The Movie Database (TMDB).

1.1. O Cliente HTTP Síncrono Moderno: Spring RestClient

No Spring Boot 4.1.x (e Spring Framework 6.1+), o RestClient é a ferramenta padrão e recomendada para efetuar chamadas HTTP síncronas a APIs externas. Ele substitui o tradicional RestTemplate (legado) e evita a complexidade reativa do WebClient quando a aplicação é baseada no modelo Servlet síncrono.

- **Configuração Única (.baseUrl & .defaultHeader):** Centraliza a URL base (https://api.themoviedb.org/3) e cabeçalhos padrão como Authorization: Bearer <TOKEN>.
- **API Fluente:** Oferece métodos expressivos como .get(), .uri(), .retrieve() e .body(Class).
- **Desserialização Automática:** O Spring integra automaticamente o Jackson para desserializar a resposta JSON externa diretamente em Records Java 21.

1.2. Anti-Corruption Layer (ACL) & Contract Ownership

Uma das regras arquiteturais fundamentais da Academia Java Backend é a preservação do Contract Ownership (Propriedade do Contrato). A nossa API local não deve expor o JSON bruto fornecido pelo TMDB nem depender das suas estruturas internas.

> **DICA DE ARQUITETURA: Anti-Corruption Layer (ACL)**
> O TMDB devolve dezenas de campos no seu JSON (como original_title, adult, popularity, vote_count, etc.). Se expuséssemos essa estrutura diretamente ao nosso cliente frontend, qualquer mudança efetuada pelo TMDB quebraria a nossa API. A Anti-Corruption Layer garante que traduzimos os DTOs Externos (TmdbMovie) para o nosso modelo de domínio interno (Movie) ou DTOs de contrato local

> (MovieCandidateResponse), isolando e protegendo a nossa aplicação contra mudanças de terceiros.

1.3. Gestão Segura de Credenciais (TMDB_TOKEN)

Chaves de API e tokens de acesso NUNCA devem ser commitados no histórico do Git. No projeto MyMovies, externalizamos o segredo utilizando a propriedade tmdb.token=${TMDB_TOKEN:} no application.properties e injetamos o token via variável de ambiente no IntelliJ IDEA.

2. Baseline Git & Estrutura Oficial de Pacotes

Garante que o teu repositório local está sincronizado com a tag do checkpoint da Aula 03 antes de iniciar:

```
git fetch --tags
git checkout checkpoint/aula-03-jpa
git checkout -b feature/aula-04-tmdb
```

A árvore de ficheiros do projeto sob o pacote pt.rumos.mymovies ficará assim estruturada:

```
pt.rumos.mymovies/
├── client/
│   └── TmdbClient.java             # [NOVO] Cliente HTTP síncrono com RestClient
├── dto/
│   ├── MovieRequest.java           # DTO de entrada local (Aula 2)
│   ├── MovieResponse.java          # DTO de saída local (Aula 2/3)
│   ├── MovieCandidateResponse.java # [NOVO] DTO de candidato retornado na pesquisa
│   └── tmdb/
│       ├── TmdbSearchResponse.java # [NOVO] DTO wrapper da pesquisa TMDB
│       └── TmdbMovie.java          # [NOVO] DTO dos detalhes do filme TMDB
├── model/
│   └── Movie.java                  # Entidade JPA (Aula 3)
├── repository/
│   └── MovieRepository.java       # Interface Spring Data JPA (Aula 3)
├── service/
│   ├── MovieService.java          # Serviço principal do catálogo
│   └── TmdbService.java           # [NOVO] Serviço de integração & ACL
└── controller/
    ├── MovieController.java       # Controller do catálogo local
    └── TmdbMovieController.java   # [NOVO] Controller para pesquisa e importação TMDB
```

3. Roteiro Guiado da Atividade Prática (Passo a Passo)

Etapa 1: Configurar Rest Client no Spring

No Spring Boot 4.1 / Spring 6, o RestClient é a alternativa síncrona moderna ao antigo RestTemplate . Ao injetarmos o RestClient.Builder no nosso TmdbClient , permitimos que a aplicação configure cabeçalhos padrão (como a *Authorization: Bearer <TOKEN>) e URLs base de forma fluente antes de criar a instância do cliente HTTP."

```
public class MyMoviesApplication {

    public static void main(String[] args) {
        SpringApplication.run(MyMoviesApplication.class, args);
    }

    // Adicione este Bean para o Spring saber como construir o RestClient.Builder
    @Bean
    public RestClient.Builder restClientBuilder() {
        return RestClient.builder();
    }
}
```

Etapa 2: Configurar application.properties e Injetar Segredo

Abre o ficheiro src/main/resources/application.properties e adiciona as propriedades do cliente TMDB:

```
# Configuração do Cliente Externo TMDB
tmdb.base-url=https://api.themoviedb.org/3
tmdb.token=${TMDB_TOKEN:}
```

No IntelliJ IDEA, acede a Edit Configurations... > Environment variables e adiciona a variável TMDB_TOKEN com o teu API Read Access Token (copiado de Settings > API no site do TMDB).

Etapa 3: Criar os DTOs Externos com Anotações Jackson

No subpacote pt.rumos.mymovies.dto.tmdb, cria os Records que representam a resposta JSON da API do TMDB. Utiliza @JsonIgnoreProperties(ignoreUnknown = true) para descartar campos não utilizados.

- **Ficheiro 1:** TmdbSearchResponse.java: Wrapper contendo a lista results.

```
package pt.rumos.mymovies.dto.tmdb;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import java.util.List;

@JsonIgnoreProperties(ignoreUnknown = true)
public record TmdbSearchResponse(
    List<TmdbMovie> results
) {}
```

- **Ficheiro 2:** TmdbMovie.java: Detalhes do candidato TMDB.

```
package pt.rumos.mymovies.dto.tmdb;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.annotation.JsonProperty;

@JsonIgnoreProperties(ignoreUnknown = true)
public record TmdbMovie(
    long id,
    String title,
    @JsonProperty("release_date") String releaseDate,
    String overview,
    @JsonProperty("vote_average") Double voteAverage
) {}
```

Etapa 4: Criar o Cliente HTTP TmdbClient com RestClient

No pacote pt.rumos.mymovies.client, cria a classe TmdbClient anotada com @Component. Injeta o RestClient.Builder e as propriedades do application.properties:

```
package pt.rumos.mymovies.client;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.HttpHeaders;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;
import pt.rumos.mymovies.dto.tmdb.TmdbMovie;
import pt.rumos.mymovies.dto.tmdb.TmdbSearchResponse;

@Component
public class TmdbClient {

    private final RestClient client;

    public TmdbClient(RestClient.Builder builder,
                      @Value("${tmdb.base-url}") String baseUrl,
                      @Value("${tmdb.token}") String token) {

       this.client= builder.baseUrl(baseUrl)
                           .defaultHeader(HttpHeaders.AUTHORIZATION, "Bearer "+ token)
                           .build();
    }

    public TmdbSearchResponse searchMovies(String query) {
        return client.get()
                .uri(uriBuilder -> uriBuilder.path("/search/movie")
                                             .queryParam("query", query)
                                             .build())
                .retrieve()
                .body(TmdbSearchResponse.class);
    }
```

```
    public TmdbMovie getMovieDetails(long tmdbId) {
        return client.get()
                .uri("/movie/{id}", tmdbId)
                .retrieve()
                .body(TmdbMovie.class);
    }
}
```

Etapa 5: Criar os DTOs Locais e o Serviço de Integração (TmdbService)

No pacote pt.rumos.mymovies.dto, cria o DTO de contrato público local MovieCandidateResponse.java:

```
package pt.rumos.mymovies.dto;

public record MovieCandidateResponse(
    long tmdbId,
    String title,
    String releaseDate
) {}
```

No pacote pt.rumos.mymovies.service, cria a classe TmdbService.java para orquestrar a busca, o mapeamento e a persistência local (Anti-Corruption Layer):

```
package pt.rumos.mymovies.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import pt.rumos.mymovies.client.TmdbClient;
import pt.rumos.mymovies.dto.MovieCandidateResponse;
import pt.rumos.mymovies.dto.MovieResponse;
import pt.rumos.mymovies.dto.tmdb.TmdbMovie;
import pt.rumos.mymovies.dto.tmdb.TmdbSearchResponse;
import pt.rumos.mymovies.model.Movie;
import pt.rumos.mymovies.repository.MovieRepository;

import java.time.LocalDate;
import java.util.List;

@Service
public class TmdbService {

    private final TmdbClient tmdbClient;
    private final MovieRepository movieRepository;

    public TmdbService(TmdbClient tmdbClient, MovieRepository movieRepository) {
        this.tmdbClient = tmdbClient;
        this.movieRepository = movieRepository;
    }

    public List<MovieCandidateResponse> searchCandidates(String query) {
```

```
        TmdbSearchResponse response = tmdbClient.searchMovies(query);
        if (response == null || response.results() == null) {
            return List.of();
        }
        return response.results().stream()
                .map(tmdb -> new MovieCandidateResponse(
                        tmdb.id(),
                        tmdb.title(),
                        tmdb.releaseDate()))
                .toList();
    }

    @Transactional
    public MovieResponse importMovie(long tmdbId) {
        TmdbMovie external = tmdbClient.getMovieDetails(tmdbId);

        int year = 2000;
        if (external.releaseDate() != null && !external.releaseDate().isBlank()) {
            year = LocalDate.parse(external.releaseDate()).getYear();
        }

        Movie movie = new Movie();
        movie.setTitle(external.title());
        movie.setDirector("Desconhecido (TMDB)");
        movie.setYear(year);
        movie.setRating(external.voteAverage()!= null ? external.voteAverage() : 0.0);

        Movie saved = movieRepository.save(movie);

        return new MovieResponse(
                saved.getId(),
                saved.getTitle(),
                saved.getDirector(),
                saved.getYear(),
                saved.getRating()
        );
    }
}
```

Etapa 6: Criar o Controller REST (TmdbMovieController)

No pacote pt.rumos.mymovies.controller, cria a classe TmdbMovieController.java expondo os endpoints /api/tmdb/movies/search e /api/tmdb/movies/{tmdbId}/import:

```
package pt.rumos.mymovies.controller;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import pt.rumos.mymovies.dto.MovieCandidateResponse;
import pt.rumos.mymovies.dto.MovieResponse;
import pt.rumos.mymovies.service.TmdbService;
```

```
import java.util.List;

@RestController
@RequestMapping("/api/tmdb/movies")
public class TmdbMovieController {

    private final TmdbService tmdbService;

    public TmdbMovieController(TmdbService tmdbService) {
        this.tmdbService = tmdbService;
    }

    @GetMapping("/search")
    public ResponseEntity<List<MovieCandidateResponse>> search(@RequestParam String
query) {
        return ResponseEntity.ok(tmdbService.searchCandidates(query));
    }

    @PostMapping("/{tmdbId}/import")
    public ResponseEntity<MovieResponse> importMovie(@PathVariable long tmdbId) {
        MovieResponse imported = tmdbService.importMovie(tmdbId);
        return ResponseEntity.status(HttpStatus.CREATED).body(imported);
    }
}
```

4. ATIVIDADE EXTRA (Testes no Bruno)

Atividade Extra A: Fluxo de Testes no Bruno Client

Abre a tua coleção no Bruno Client e executa o fluxo completo de integração:

- **1. Pesquisar candidatos:** GET http://localhost:8080/api/tmdb/movies/search?query=Interstellar -> Confirma o retorno 200 OK e anota o tmdbId de um dos candidatos.
- **2. Importar filme por tmdbId:** POST http://localhost:8080/api/tmdb/movies/157336/import -> Confirma o retorno 201 Created e o JSON do filme importado com ID local.
- **3. Validar Persistência no Catálogo:** GET http://localhost:8080/api/movies -> Confirma que o filme importado do TMDB já consta no banco de dados H2!

5. Checklist de Conclusão & Git Tag Checkpoint

Antes de concluir a aula, confirma se o teu projeto atende a todos os critérios da Definition of Done:

- **[ ]** A variável TMDB_TOKEN está externalizada e não aparece no código-fonte.
- **[ ]** O TmdbClient utiliza o RestClient síncrono com o prefixo Bearer Token.
- **[ ]** A pesquisa GET /api/tmdb/movies/search?query=... devolve os candidatos sem alterar o banco de dados.
- **[ ]** A importação POST /api/tmdb/movies/{tmdbId}/import obtém os detalhes, mapeia a data para ano e grava no H2 com o status 201 Created.
- **[ ]** Os DTOs do TMDB estão totalmente isolados da camada local pela Anti-Corruption Layer.

Com todos os testes validados no Bruno, executa os comandos no terminal para registar o checkpoint oficial da Aula 04:

```
git add .
git commit -m "feat: implement tmdb integration via RestClient and ACL"
git checkout main
git merge feature/aula-04-tmdb
git tag checkpoint/aula-04-tmdb
git push origin main –tags
```